

APUNTS LAB2 GRÀFICS

Geometry shaders:

- està entre vertex shader i fragment shader
- rep i dona primitives, les envia a primitives assembler
- out del vertex va cap al geometry, i del geometry pot anar al fragment
- sortida del geometry en clipping, el vertex pot calcular el que vulgui
- donem valor, emitVert(), donem valor

`vec3 baricentre = (vec3(v0) + vec3(v1) + vec3(v2)) / 3.0;`

`vec3 punt = step * round(baricentre / step);`

`float ms = step / 2.0;`

· area:

- > producte vectorial equival a `vec3 res = cross(u, v)`
- > mòdul del vector equival a `length(res)`
- > serveix per anar amb compte de quins vectors utilitzar

· explode:

- > serveix per veure com funciona els out del VS al in del GS, fer in `vec3 normals[]`

· lego:

- > serveix per veure com utilitzar textures + fer noves primitives
- > fer un quadrat cal utilitzar l'step + baricentre

· shadow:

- > serveix per veure com fer ombres simples
- > crea dos triangles, un per l'objecte i l'altre per l'ombra

· shadow2:

- > mateix que abans, però ara amb un terra (2 triangles posats al inici)

· rgb color space:

- > permet saber com crear cubs i com calcular colors i passar paràmetres

· grass:

- > saber com utilitzar textures

· rubik:

- >

Tipus de Plugins:

- **Effect Plugin:** plugin que implementa els mètodes **preFrame** i/o **postFrame** (a banda d'altres mètodes que pugueu necessitar). Exemple a **effectcrt**.
- **Draw Plugin:** plugin que implementa el mètode **drawScene** (a banda d'altres mètodes que pugueu necessitar). Exemple a **drawbong**. Gestionar transferència de informació entre GPU i CPU i s'encarrega de cridar a pintar (fer VAO i VBO).
- **Action Plugin:** plugin que implementa mètodes d'entrada com ara **keyPressEvent** (a banda d'altres mètodes que pugueu necessitar). Teniu un exemple a **navigatedefault**.
- **Render Plugin:** plugin que implementa el mètode **paintGL** (a banda d'altres mètodes que pugueu necessitar). Teniu un exemple a **renderdefault**.

Effect Plugins:

Exemple de effect plugin per a posar uniforms:

```
void AnimateVertices::onPluginLoad()
{
    glwidget()->makeCurrent();
    // Vertex shader
    vs = new QOpenGLShader(QOpenGLShader::Vertex, this);
    vs->compileSourceFile("animate-vertices1.vert"); //carreguem arxiu
    cout << "VS log:" << vs->log().toString() << endl;
    // Fragment Shader
    fs = new QOpenGLShader(QOpenGLShader::Fragment, this);
    fs->compileSourceFile("animate-vertices1.frag"); // <-- Cargamos desde el archivo
    cout << "FS log:" << fs->log().toString() << endl;
    // Program
    program = new QOpenGLShaderProgram(this);
    program->addShader(vs); program->addShader(fs);
    program->link();
    cout << "Link log:" << program->log().toString() << endl;
    // Timer
    elapsedTimer.start(); //comença crono
    timer = new QTimer(this);
    // envia timeout quan 0 receptor la acció a fer
    connect(timer, SIGNAL(timeout()), glwidget(), SLOT(update()));
    timer->start(1000);
}
```

//uniforms i tal

```
void AnimateVertices::preFrame()
{
    program->bind(); //activem
    QMatrix4x4 MVP = camera()->projectionMatrix() * camera()->viewMatrix();
    program->setUniformValue("modelViewProjectionMatrix", MVP);
    program->setUniformValue("amplitude", 0.1f); //f per no donar error ambiguetat
    program->setUniformValue("freq", 1);
    QMatrix3x3 NM = camera()->viewMatrix().normalMatrix(); //qt ens fa el calcul de normalMatrix
    program->setUniformValue("normalMatrix", NM);
    program->setUniformValue("time", float(elapsedTimer.elapsed()) / 1000.0f); //en segons
}
```

Draw Plugins:

Action Plugins: